AGENDA 5

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE II

Expediente

Autores:

TIAGO ANTONIO DA SILVA

Revisão Técnica:

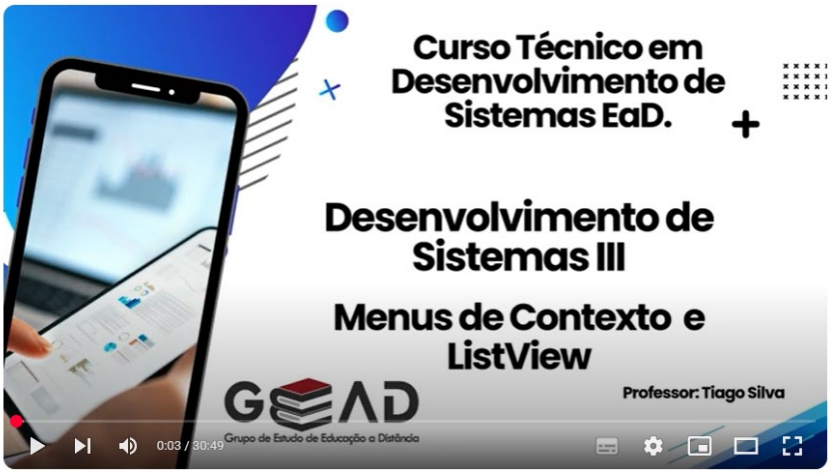
KELLY CRISTIANE DE OLIVEIRA DAL POZZO

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

Avançando no desenvolvimento do Aplicativo Minhas Compras nesta agenda vamos abordar o funcionamento da navegação entre telas a partir de um item selecionado na **ListView**, menu de contexto de ações da ListView e vamos relembrar a DisplayAlert com a mensagem de confirmação.

Assista o vídeo apresentando o passo a passo para criação do aplicativo:



Disponível em: https://www.youtube.com/watch?v=Zs4vlFkNtSI

Agora vamos compreender o passo a passo apresentado no vídeo.

Conceito Try-Catch

No módulo 1 do curso foi apresentado de forma breve o conceito de tratamento de exceções, este conceito é comum em diversas linguagens de programação.

O bloco **try-catch** é um mecanismo de tratamento de exceções, permitindo que o programa continue funcionando mesmo quando um erro inesperado ocorre, evitando que o aplicativo trave ou feche de forma inesperada. O comando try tenta executar um bloco de código normalmente, caso ocorra um erro dentro do bloco de código, o catch captura esse erro e executa um código alternativo para tratar a situação.

Vamos a um exemplo simples:

```
1   try
2   {
3       int resultado = 10 / 0; // Isso pode causar um erro (divisao por zero)
4       Console.WriteLine($"Resultado: {resultado}");
5   }
6   catch (Exception ex)
7   {
8       Console.WriteLine($"Ocorreu um erro: {ex.Message}");
9   }
10
```

No exemplo apresentado acima, o bloco try tenta executar a divisão 10/0, mas como não é possível dividir por zero, um erro acontece. Neste momento o bloco catch é acionado, captura esse erro e exibe uma mensagem para o usuário, em vez de fazer o programa.

No projeto App Minhas Compras, o try-catch está utilizado várias vezes, como por exemplo, para evitar falhas ao carregar os produtos do banco de dados.

```
protected async override void OnAppearing()
{
    try
    {
        lista.Clear();
        List<Produto> tmp = await App.Db.GetAll();
        tmp.ForEach(i => lista.Add(i));
    }
    catch (Exception ex)
    {
        await DisplayAlert("Ops", ex.Message, "OK");
    }
}
```

No trecho de código apresentado, o programa tenta buscar os produtos do banco App.Db.GetAll(). Se o banco não estiver acessível ou outro erro ocorrer, o catch exibe um alerta “Ops” com a mensagem do erro.

Menu de Contexto

No .NET MAUI, é possível adicionar um Menu de Contexto a uma ListView usando o comando ContextActions. Isso permite a interação dos usuários com os itens da lista por meio de ações como Excluir, Editar ou Compartilhar.

A forma mais simples de adicionar um Menu de Contexto a um item de uma ListView é utilizando ContextActions dentro de um ViewCell.

Vamos analisar um exemplo apresentado no projeto:

```
<ListView x:Name="lst_produtos" ItemSelected="lst_produtos_ItemSelected">
    <ListView.ItemTemplate>
        <DataTemplate>
            <ViewCell>
                <ViewCell.ContextActions>
                    <MenuItem Text="Excluir" Clicked="MenuItem_Clicked" IsDestructive="True" />
                </ViewCell.ContextActions>
                <StackLayout Padding="10">
                    <Label Text="{Binding Nome}" FontSize="18" />
                </StackLayout>
            </ViewCell>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

Onde o comando **MenuItem** define as opções do menu de contexto, neste caso o excluir, já o comando **Clicked** especifica o evento chamado ao clicar no item. Por fim, o IsDestructive="True", destaca o botão como uma ação destrutiva, como por exemplo: excluir.

Confirmações com DisplayAlert

Como já visto anteriormente, o método **DisplayAlert** é uma ferramenta utilizada para exibir mensagens ao usuário, pedir confirmação ou fornecer avisos. Ele pode ser usado para criar caixas de diálogo que exigem interação antes de continuar uma ação: se encaixa perfeitamente com o item anterior, onde adicionamos o menu de contexto para realizar a exclusão de itens vistos na **ListView**.

```
bool resposta = await DisplayAlert("Título", "Deseja continuar?", "Sim", "Não");
```

Onde:

- **Título:** Texto exibido no cabeçalho do alerta.
- **Mensagem:** Texto principal do alerta.
- **Botão de Confirmação:** Texto para a opção positiva (exemplo: "Sim").
- **Botão de Cancelamento:** Texto para a opção negativa (exemplo: "Não").

O método apresentado retorna um valor booleano (bool), sendo true se o usuário selecionar a opção de confirmação e false caso escolha cancelar.

Para utilizar o DisplayAlert, é importante lembrar que:

- *O DisplayAlert só pode ser chamado a partir de um método async, pois ele usa await.*
- *Deve ser usado com moderação para não interromper excessivamente o fluxo do usuário.*

Processando o Evento ItemSelected de uma ListView

O evento **ItemSelected** da **ListView** permite detectar quando um item é selecionado e, com isso, é possível navegar para uma nova página enviando os dados do item escolhido via **BindingContext**. Assim se caso a o item da lista tenha o objeto completo que é desejado ver em detalhes, ao trocar de páginas, recuperamos o contexto e podemos exibir os dados para o usuário.

Com esse método é possível navegar entre telas e passar dados do item selecionado de forma limpa e eficiente. Isso melhora a usabilidade e mantém um código bem estruturado.

Esse recurso foi utilizado no projeto para ver a página de detalhes de cada item e se for caso realizar uma edição.

Detalhando o Método EditarProduto

Para melhor compreensão do desenvolvimento do projeto, vamos detalhar o método **EditarProduto.**

O método EditarProduto identificado com **ToolbarItem_Clicked** é responsável por editar os dados de um produto já cadastrado no banco de dados. Esse método é acionado quando o usuário clica no botão de Salvar na barra de ferramentas.

Após a assinatura do método é acionado o objeto **BindingContext** que contém os dados do produto que foi selecionado na tela anterior, garantindo que os valores editados sejam salvos no mesmo registro, e não em um novo.

```
        Produto produto_anexado = BindingContext as Produto;
```

Em seguida é criado o objeto **Produto** com os novos valores.

```
Produto p = new Produto
        {
            Id = produto_anexado.Id,
            Descricao = txt_descricao.Text,
            Quantidade = Convert.ToDouble(txt_quantidade.Text),
            Preco = Convert.ToDouble(txt_preco.Text)
        };
```

O método **Update(p)** chama a classe de banco de dados SQLite, garantindo que os dados do produto sejam atualizados no banco.

```
        await App.Db.Update(p);
```

Após a atualização, um alerta de confirmação é exibido para o usuário.

```
await DisplayAlert("Sucesso!", "Registro Atualizado", "OK");
```

Após salvar os dados, é feito o redirecionamento para a tela anterior (ListaProduto.xaml).

```
await Navigation.PopAsync();
```

Toda a codificação foi realizada dentro do tratamento de exceções Try-Catch, assim caso ocorra algum erro, a aplicação não trava, em vez disso, o erro é capturado e exibido para o usuário de forma amigável.

```
private async void ToolbarItem_Clicked(object sender, EventArgs e)
{
    try
    {
        Produto produto_anexado = BindingContext as Produto;

        Produto p = new Produto
        {
            Id = produto_anexado.Id,
            Descricao = txt_descricao.Text,
            Quantidade = Convert.ToDouble(txt_quantidade.Text),
            Preco = Convert.ToDouble(txt_preco.Text)
        };

        await App.Db.Update(p);
        await DisplayAlert("Sucesso!", "Registro Atualizado", "OK");
        await Navigation.PopAsync();
    }
    catch (Exception ex)
    {
        await DisplayAlert("Ops", ex.Message, "OK");
    }
}
```

CÓDIGO-FONTE DA AGENDA:

https://github.com/tiagotas/MauiAppMinhasCompras/tree/4ee543ada3a0411dbae3d3130db2403dcb024644